

Semantic Query Compressor

Python Project Assignment Report

Keywords : AMR-Inspired Graph Compression, LLM Evaluation, Similarity Scoring

Project at a Glance	
Purpose	Compress long queries into shorter ones; verify LLM responses remain consistent
Core Outputs	compressed_query original_output compressed_output similarity_score
Compression Modes	AMR-inspired graph compression (default) or LLM-based rewriting
LLM Backend	Groq API (configurable model, temperature = 0.0)
Similarity Scorer	TF-IDF cosine similarity between original & compressed LLM responses
Runtime Requirement	Python 3.10+ Groq API key Llam 3.1 8b model [LLM evaluator]

1. Project Overview

Semantic Query Compressor is a pipeline that takes a verbose text query (100+ words), reduces it to a shorter, semantically equivalent query, and then checks whether a Large Language Model (LLM) responds consistently to both versions. In this project, we implemented **Abstract Meaning Representation(AMR)** Inspired Graph Compression Algorithm to compress long queries to short queries. This validates that the compressed prompt preserves the original intent.

1.1 Four Core Outputs

- compressed_query — the shorter query derived from the original.
- original_output — LLM Evaluator response to the original long query.
- compressed_output — LLM Evaluator response to the compressed query.
- similarity_score — a bounded [0, 1] score comparing the two LLM responses.

In our project we have used Llama 3.1 8b for LLM evaluator.

2. System Architecture

The pipeline has three sequential stages: compression, LLM evaluation, and similarity scoring. Each stage is independently configurable.

Stage 1	COMPRESSION — Sentences are scored by our AMR graph inspired custom query compression algorithm and selected using MMR.
Stage 2	LLM EVALUATION — Both original and compressed queries are sent to Llama 3.1 8b model(through Groq API) with identical settings (model, temperature = 0.0, system prompt).
Stage 3	SIMILARITY SCORING — TF-IDF cosine similarity is computed between the two LLM responses to produce a bounded [0,1] score.

2.1 File Structure

```
semantic_query_compressor/  
|-- main.py # entry point  
|-- example.txt # Sample input  
  
|-- README.md # describision of the project  
|-- requirements.txt #  
|-- requirements-dev.txt  
|-- .env.example  
|-- src/  
|   |-- semantic_query_compressor/  
|       |-- __init__.py  
|       |-- cli.py # CLI interface  
|       |-- compressor.py # AMR graph compression logic  
|       |-- llm.py # Groq API client used to call Llama 3.1  
|       |-- pipeline.py # Orchestrates all three stages  
|       |-- similarity.py # TF-IDF cosine scorer  
|       |-- text_utils.py # Tokenization helpers  
|-- tests/  
|   |-- test_pipeline.py # Offline tests (no Groq calls for safety)
```

3. Compression Algorithm (AMR-Inspired Graph)

The default compressor is an extractive algorithm inspired by Abstract Meaning Representation (AMR). It never generates new text — the compressed query is always composed of sentences that already appear in the original. The algorithm proceeds in five phases.

Phase 1 — Sentence Splitting & Concept Extraction

The query Q is split into individual sentences. Each sentence S_i is tokenised and deduplicated to produce a concept set C_i (stop words and punctuation are removed).

$$Q = \{S_1, S_2, \dots, S_n\} \quad C_i = \text{UniqueTokens}(S_i)$$

Example: for $S_1 = \text{'Design a Python function to summarise text efficiently'}$, the concepts are $\{\text{design, python, function, summarise, text, efficiently}\}$.

Phase 2 — Concept Graph Construction

A weighted undirected graph $G = (V, E)$ is built where each node is a unique concept. Edge weights capture two types of relationship:

- Adjacent words in the same sentence share an edge with weight +2 (stronger signal).
- Co-occurring (non-adjacent) words share an edge with weight +1 (weaker co-occurrence).

Highly connected concepts are treated as semantic hubs — they recur across many sentences and carry the most meaning.

Phase 3 — Concept Importance Scoring

Weighted degree centrality is computed for every concept, then normalised:

$$\text{deg}(c) = \sum w(c, n) \text{ for all neighbours } n \quad \text{score}(c) = \text{deg}(c) / \max_c' \text{deg}(c')$$

Concepts that appear in many sentences or sit between adjacent important words receive high scores. This produces a value in $[0, 1]$ per concept.

Phase 4 — Sentence Scoring with Bonus

Each sentence is scored by summing its concept scores, normalised by the square root of concept count (to avoid length bias), plus a bonus for task/constraint language which is detected using a custom task and constraint keyword list:

$$\text{score}(S_i) = (\sum \text{score}(c) \text{ for } c \text{ in } C_i) / \sqrt{|C_i|} + \text{bonus}(S_i)$$

Sentence Type	Trigger Keywords	Bonus
Task sentence	"design", "create", "build"	+1.00

Constraint sentence	"avoid", "must", "ensure"	+0.75
Other	—	0

Phase 5 — MMR Selection

Sentences are selected greedily using *Maximal Marginal Relevance (MMR)*. This balances relevance against redundancy to avoid selecting near-duplicate sentences:

$$\text{MMR} = \lambda \cdot \text{importance} - (1-\lambda) \cdot \text{redundancy} \quad [\lambda = 0.75]$$

Sentence similarity is measured with *Jaccard similarity*: $\text{sim}(A, B) = |A \cap B| / |A \cup B|$. The selected sentences are reordered to match their original position in the query, then concatenated to form Q'.

The algorithm optimizes:

$\max(\text{Importance}) - (\text{Redundancy})$
while preserving task and constraint information.

4. Worked Example on 3 sentences

Input — three sentences describing a Python function requirement:

S1: Design a Python function to summarize text efficiently.
 S2: The function should avoid redundancy and preserve meaning.
 S3: It should also avoid repeating information unnecessarily.

Step-by-step scoring workout

Sent.	Concepts	Concept Sum	$/ \sqrt{ C }$	Final Score (by adjusting with bonus)
S1	design, python, function, summarize, text, efficiently	2.7	$2.7/\sqrt{6} \approx 1.10$	$1.10 + 1.0 = 2.10$ (task)
S2	function, avoid, redundancy, preserve, meaning	3.5	$3.5/\sqrt{5} \approx 1.56$	$1.56 + 0.75 = 2.31$ (constraint)
S3	avoid, repeating, information, unnecessarily	2.1	$2.1/\sqrt{4} = 1.05$	$1.05 + 0.75 = 1.80$ (constraint)

MMR Selection

Ranking: S2 (2.31) > S1 (2.10) > S3 (1.80)

- Iteration 1: S2 selected (highest score).
- Iteration 2: $\text{sim}(S2, S1) \approx 0.10$, $\text{sim}(S2, S3) \approx 0.33 \rightarrow \text{MMR}(S1) = 0.75(2.10) - 0.25(0.10) \approx 1.55$ wins. S1 selected.

- Iteration 3: S3 is too similar to S2 (constraint overlap) → dropped.

Final Output

$Q' = \{S1, S2\}$

Compressed: 'Design a Python function to summarize text efficiently while avoiding redundancy and preserving meaning.'

Retained: task intent (design), core action (summarize text), constraints (avoid redundancy, preserve meaning). S3 was pure redundancy.

5. Evaluation Metrics

Five metrics used for evaluation.

Metric	Formula	What it tells you
Compression Ratio	$CR = Q' / Q $	Fraction of words kept; lower = more compressed
Retention Ratio	$\text{words}(Q') / \text{words}(Q)$	Alias for CR, shown in demo output
Critical Preservation	$CPS = \text{avg max sim}(s, s') \text{ over critical } s$	How well task/constraint sentences survived
Query Similarity	$\text{TF-IDF cosine}(Q, Q')$	Lexical overlap between original and compressed query
LLM Response Sim.	$\text{TF-IDF cosine}(R_{\text{orig}}, R_{\text{comp}})$	Core metric: are LLM answers consistent?
Redundancy Reduction	$RR = 1 - \text{avg pairwise sim in } Q'$	How non-repetitive the compressed output is

5.1 Composite Score

A weighted combination for holistic evaluation:

$$\text{Score} = \alpha \cdot IR + \beta \cdot CPS + \gamma \cdot RR - \delta \cdot CR$$

IR = Information Retention, CPS = Critical Preservation Score, RR = Redundancy Reduction, CR = Compression Ratio. $\alpha, \beta, \gamma, \delta$ are weighting coefficients chosen per use-case.

6. Setup & Usage

6.1 Installation

```
cd semantic_query_compressor
python -m venv .venv && source .venv/bin/activate # or Activate.ps1 on Windows
pip install -r requirements-dev.txt && pip install -e .
cp .env.example .env # then add GROQ_API_KEY and GROQ_LLM_MODEL
```

6.2 Python API

```
from semantic_query_compressor import compress_and_evaluate

result = compress_and_evaluate(query, compression_method='amr')
print(result.compressed_query, result.similarity_score)
print(result.as_dict())
```

6.3 CLI (Command Line Interface) pipeline

```
semantic-query-compressor --file example.txt --pretty
semantic-query-compressor --file example.txt --target-ratio 0.25 --compression-method llm --pretty
```

7. Limitations & Suggested Improvements

7.1 Current Limitations

- Not fully AMR: this is an AMR-inspired approximation; not formal AMR parser is used.
- TF-IDF similarity is lexical — two semantically identical answers with different vocabulary will score low.
- Dense prompts with many distinct requirements compress less aggressively (by design)
- More fine-tuned task keywords and constraint keywords should be used.
- LLM-based compression may rewrite text, so `compressed_query` may not be an exact subset of the original.
- As a fallback logic for safety a LLM query generator is created.

8. Conclusion

Non-LLM-based compression methods, such as rule-based or graph-based approaches, often struggle to preserve the full semantic meaning of a query. Because they rely on surface features like word frequency, co-occurrence, or predefined rules, they lack deep contextual understanding. As a result, they may remove or distort important nuances, implicit relationships, or constraints within the text. This can lead to compressed outputs that are shorter but semantically incomplete or misleading compared to the original input. In this project we propose a **constraint-aware, graph-based compression strategy** to deal with this problem,

9. References

- Mihalcea, R., & Tarau, P. (2004). *TextRank: Bringing order into text*. Proceedings of EMNLP.
- Harris, Z. (1954). *Distributional structure*. Word, 10(2–3), 146–162.
- Carbonell, J., & Goldstein, J. (1998). *The use of MMR for diversity-based reranking for reordering documents and producing summaries*. SIGIR.
- Salton, G., Wong, A., & Yang, C. (1975). *A vector space model for automatic indexing*. Communications of the ACM, 18(11), 613–620.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of deep bidirectional transformers for language understanding*. arXiv:1810.04805.

End of Report